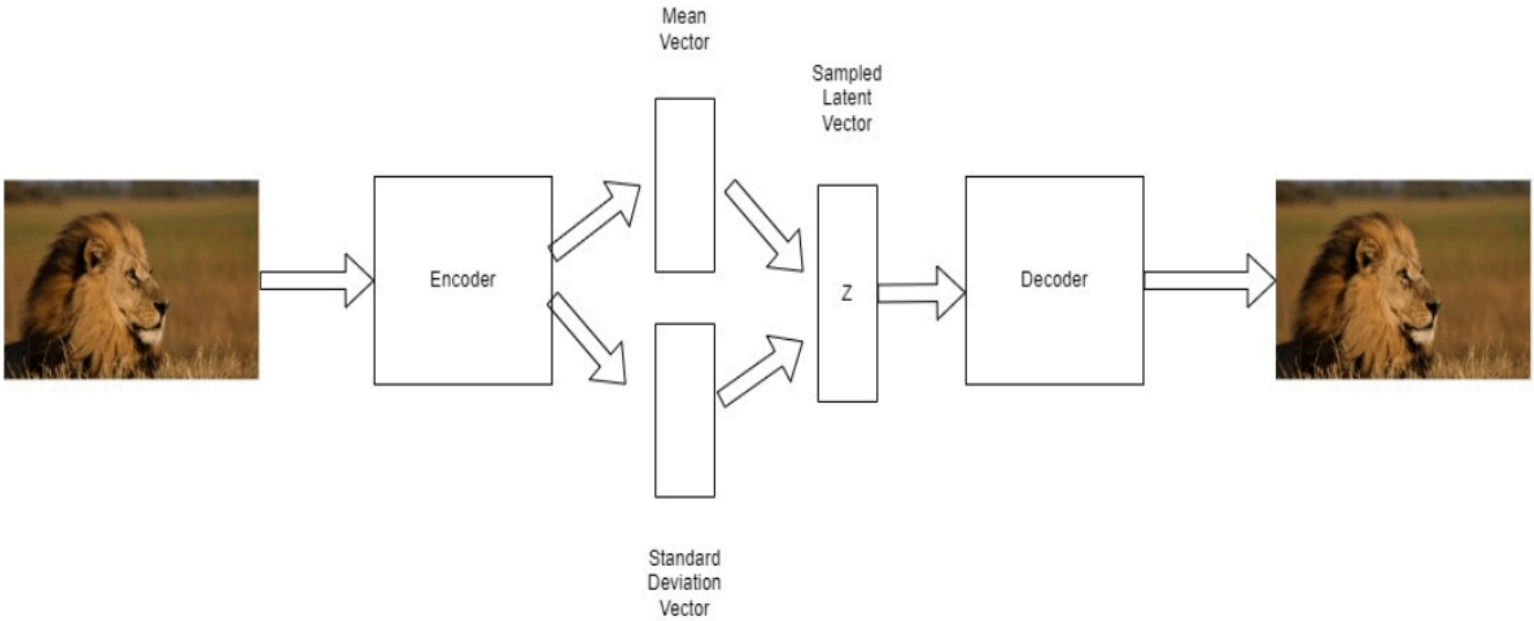
Module-3
Variational autoencoder

- A variational autoencoder can be defined as being an autoencoder whose training is regularized to avoid overfitting and ensure that the latent space has good properties that enable generative process.
- The VAE is programmed to minimize the dissimilarity between the original and reconstructed data, enabling it to comprehend the underlying data distribution and generate new samples that conform to the same distribution.
- One notable advantage of VAEs is their ability to generate new data samples resembling the training data.
- Because the VAE's latent space is continuous, the decoder can generate new data points that seamlessly interpolate among the training data points

Basic Components of VAEs

Variational Autoencoders (VAEs) are a class of deep learning models used for learning latent representations of data, and they are particularly popular in unsupervised learning tasks.



A VAE consists of several basic components:

- **Encoder Network:**
  - This component takes the input data and encodes it into a latent space.
  - Typically, the encoder network is a **neural network** that compresses the input into a lower-dimensional representation.
  - In the context of a VAE, the encoder outputs parameters to a probability distribution—usually a **Gaussian distribution**—representing the data in latent space.
  - The **output** is often the **mean (μ)** and **standard deviation (σ)** of this distribution.

- **Latent Space:**
  - This is a typically lower-dimensional space where the input data is represented in a compressed form.
  - In VAEs, this space is probabilistic, meaning that each point in the latent space does not correspond to a single state, but rather a distribution over possible states.
  - This probabilistic nature allows for generating new data points.

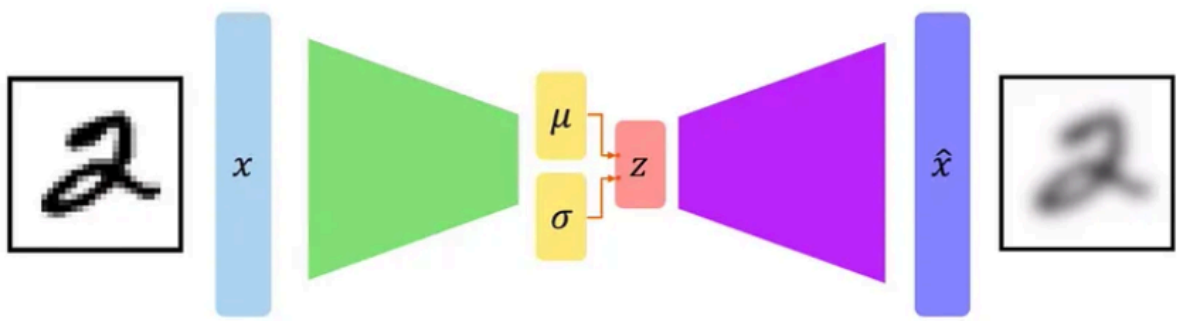
- **Reparameterization Trick:**
  - In order to make the model differentiable and allow for backpropagation, the reparameterization trick is used.
  - Instead of sampling directly from the distribution output by the encoder, the model samples from a standard normal distribution and then re parameterizes the sample using the mean and standard deviation provided by the encoder.
  - This allows for the gradient of the loss function to be back propagated through the random sampling process.

- **Decoder Network:**
  - The decoder takes the latent representation and **reconstructs the input** data from it.
  - Like the encoder, the decoder is typically a **neural network,** but it works in the opposite direction, mapping the latent space back to the original data space.
  - The goal of the decoder is to reconstruct the input data as accurately as possible.

- **Loss Function:**
  - The loss function in VAEs has two main components:

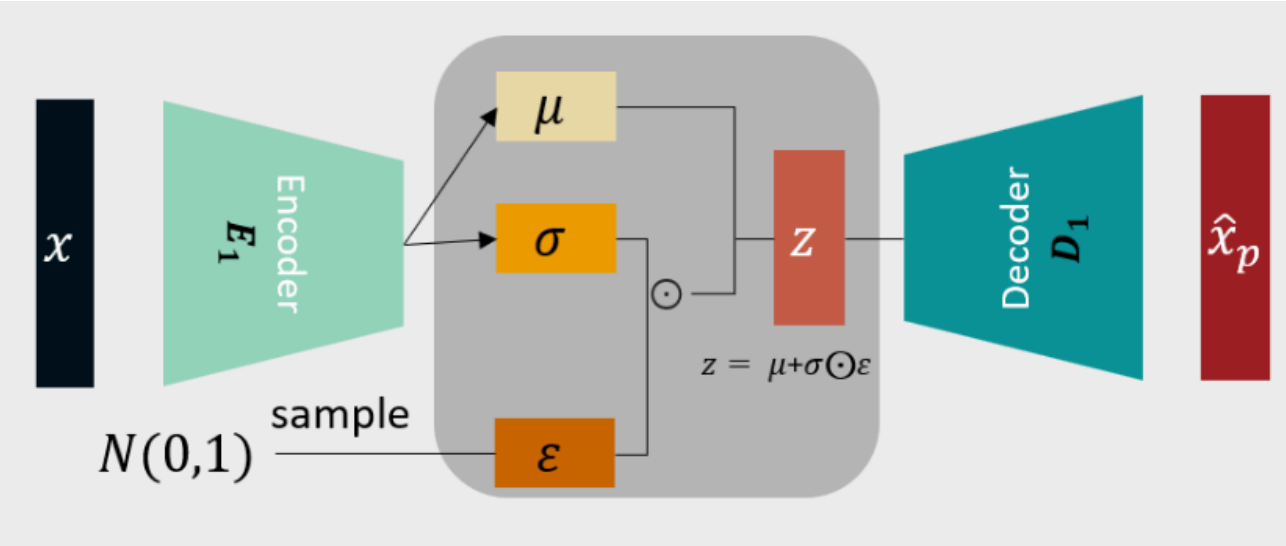
- **Reconstruction Loss:**
    - This measures how well the decoder is able to reconstruct the input data from the latent representation.
    - It encourages the decoder to learn to accurately reconstruct the original data.
- **Regularization Term (KL Divergence):**
    - This part of the loss function ensures that the distribution of the latent variables stays close to a prior distribution (usually a standard normal distribution).
    - It prevents the model from encoding the data in an arbitrary way in the latent space, encouraging generalization and improving the ability to generate new data points.

These components work together in a VAE to learn efficient representations of data and to enable the generation of new data points that are similar to the original dataset. VAEs have been widely used in various applications such as image generation, anomaly detection, and more.



What is the reparameterization trick?

- The reparameterization trick is a mathematical operation used in the training process of VAEs.
- It's a technique that helps bypass a significant problem in training VAEs: the backpropagation algorithm cannot be applied directly through random nodes.
- The VAE architecture includes a sampling operation where we sample latent variables from a distribution parameterized by the outputs of the encoder.
- The direct application of backpropagation here is problematic because of the inherent randomness of the sampling operation.
- In other words, computing the latent z involves sampling from a (multivariate normal) distribution.
- This sampling operation introduces stochasticity and therefore cannot be differentiated.



VAE illustration. The gray part of the model is the reparameterization trick to compute the latent z

Note: The reason this operation is stochastic, or random, is because drawing a sample from a probability distribution is a random process. Even though the parameters of the distribution (the mean and variance) are fixed output of the encoder for a given input, the actual samples that you draw from the distribution will vary each time you draw a sample. This is the nature of sampling from a probability distribution.

Why Do We Need the Reparameterization Trick?

- Let's consider once again the structure of a VAE: it comprises an encoder, a decoder, and a latent space in between.
- The encoder's role is to map inputs to a distribution in the latent space, defined by two parameters: a mean (μ) and a standard deviation (σ).
- This latent distribution represents the learned representation of the input data.
- The decoder then generates an output by sampling points from this distribution.
- However, if we were to sample these points directly, this operation introduces a stochastic element that prevents the direct application of backpropagation.
- Backpropagation relies on computing gradients of deterministic (i.e., non-random) operations.
- Therefore, we need a method that introduces the necessary randomness for sampling while preserving the differentiability of the operations involved.
- This is where the reparameterization trick comes in.

- ➔ The backpropagation algorithm, which is used to train neural networks, requires the ability to compute exact gradients.
- ➔ Because the sampling operation is random, it doesn't have a well-defined gradient (weights always randomly changing).
- ➔ This means that we can't use the backpropagation algorithm to train the encoder and decoder networks in a VAE.

The Reparameterization Trick Unveiled

- The reparameterization trick works by separating the deterministic and the stochastic parts of the sampling operation.
- Instead of directly sampling from the distribution $N(\mu,\sigma^2)$, we sample ε from a standard Normal distribution $N(0,1)$ and compute the desired sample z as:

  $$Z = \mu+\sigma*\varepsilon$$

  Here,
  - ε introduces the necessary randomness.
  - The operation $\mu+\sigma*\varepsilon$ is entirely deterministic and differentiable, meaning we can apply backpropagation through it.

- This method allows us to incorporate the random element required for sampling from the latent distribution while preserving the chain of differentiable operations needed for backpropagation.

Difference between autoencoder and variational autoencoder

The Encoder

In an autoencoder, each image is mapped directly to one point in the latent space. In a variational autoencoder, each image is instead mapped to a multivariate normal distribution around a point in the latent space, as shown in Figure 3-11.

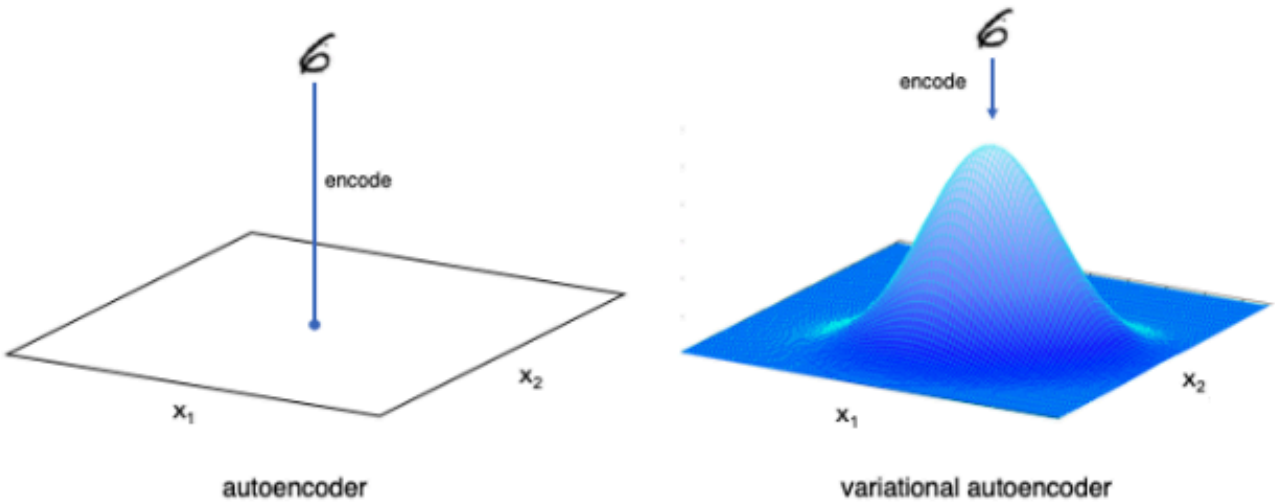
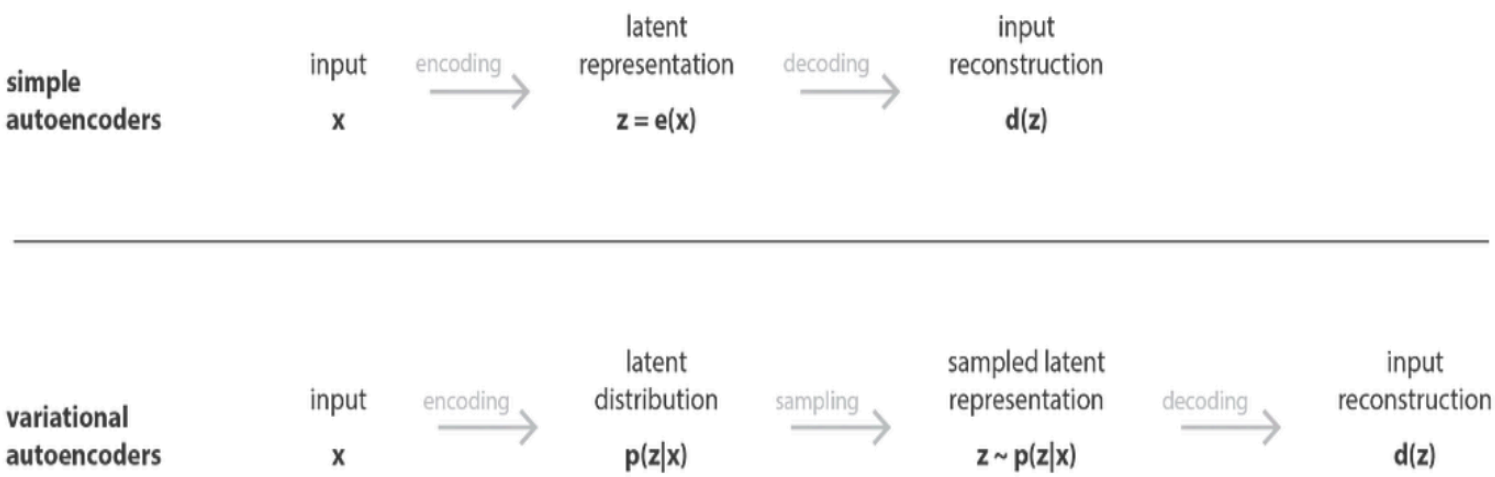


Figure 3-11. The difference between the encoder in an autoencoder and a variational autoencoder

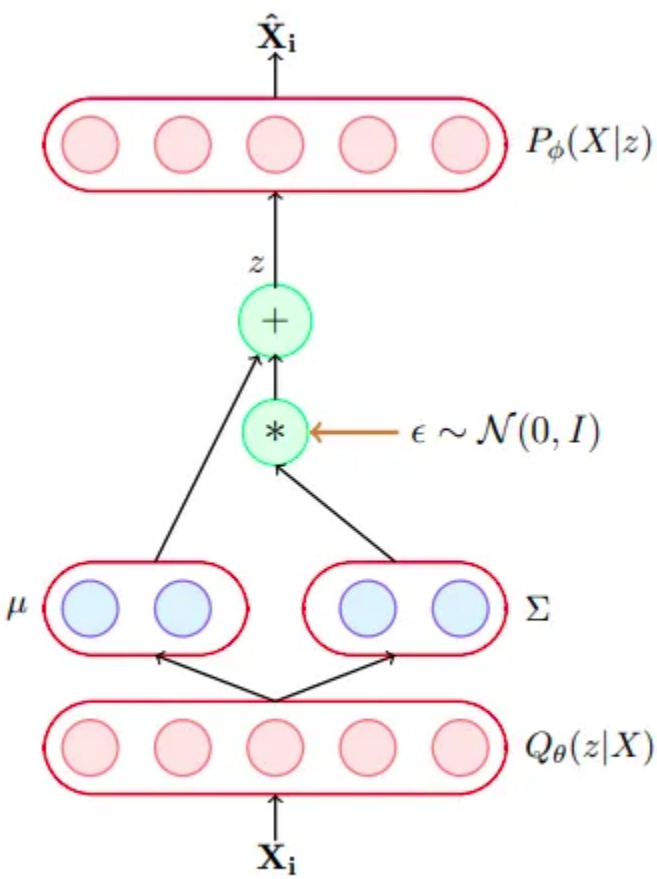


Difference between autoencoder (deterministic) and variational autoencoder (probabilistic).

Architecture of VAEs

- A VAE typically has two major components: An encoder connection and a decoder connection.
- **Encoder**:
    - An encoder network transforms The input data into a low-dimensional secret space, often called a “secret code”.
    - Various neural network topologies, such as fully connected or convolutional neural networks, can be investigated for implementing encoder networks.
    - The architecture chosen is based on the characteristics of the data.
    - The encoder network produces essential parameters, such as the mean and variance of a Gaussian distribution, necessary for sampling and generating the latent code.

- **Decoder**:
    - Similarly, researchers can construct the decoder network using various types of neural networks, and its objective is to reconstruct the original data from the provided latent code.



Architecture of VAE

Encoder Network:

- **Input**: High-dimensional data (like images, text, etc.).
- **Function**: Compresses the input into a lower-dimensional latent space.
- **Output**: Two vectors representing the mean (μ) and logarithm of the variance ($\log(\sigma^2)$) of a Gaussian distribution.

Latent Space:

- **Representation**: A compressed, probabilistic representation of the input data.

Reparameterization Trick:

- **Function**: Enables backpropagation through a stochastic process.
- **Process**: A sample `z` is drawn from the Gaussian distribution defined by μ and σ, using `z = μ + ε * σ`, where ε is a random normal variable. This allows gradients to flow through μ and σ.

Decoder Network:

- **Input**: Sampled points `z` from the latent space.
- **Function**: Reconstructs the input data from the latent space representation.
- **Output**: Reconstructed data, aiming to be as close to the original input as possible.

Loss Function:

- **Reconstruction Loss**: Measures the difference between the original input and its reconstruction (e.g., Mean Squared Error for continuous data, Binary Cross-Entropy for binary data).
- **KL Divergence**: Regularization term ensuring the distribution of latent variables stays close to a standard normal distribution.

Training of VAEs

Forward Pass:

- Pass input through the encoder to get μ and $\log(\sigma^2)$.
- Apply the reparameterization trick to sample from the latent space.
- Pass the sampled latent vector through the decoder to get the reconstructed output.

Loss Calculation:

- Compute the reconstruction loss between the input and the reconstructed output.

- Calculate the KL Divergence between the learned distribution in the latent space and the standard normal distribution.
- The total loss is the sum of these two components.

Backward Pass and Optimization:

- Backpropagate the total loss through the entire network (encoder and decoder).
- Update the weights using an optimization algorithm (like Adam).

Regularization and Overfitting:

- VAEs inherently help prevent overfitting through their regularization term (KL Divergence), but additional techniques (like dropout, early stopping) can be used.

Iterations:

- This process is repeated for several epochs over the training dataset.

Evaluation and Generation:

- After training, the model can be evaluated on how well it reconstructs unseen data.
- It can also generate new samples by feeding random points from the latent space into the decoder.

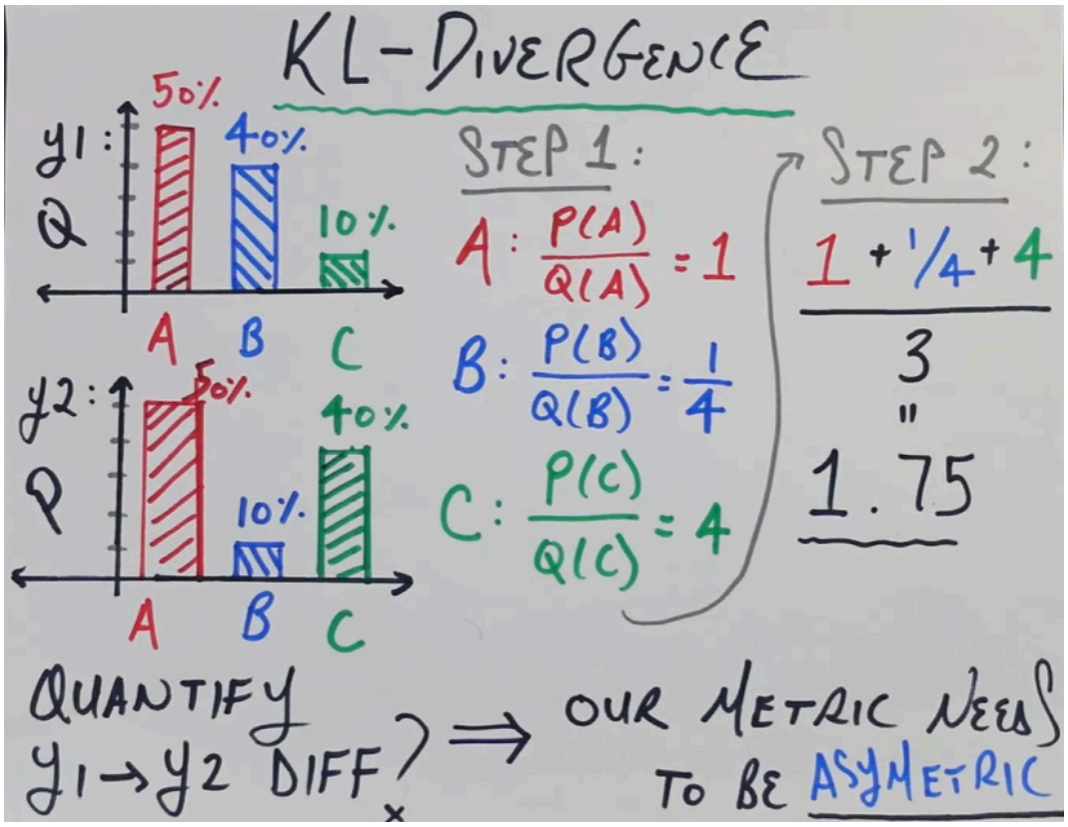
Key Points in Training

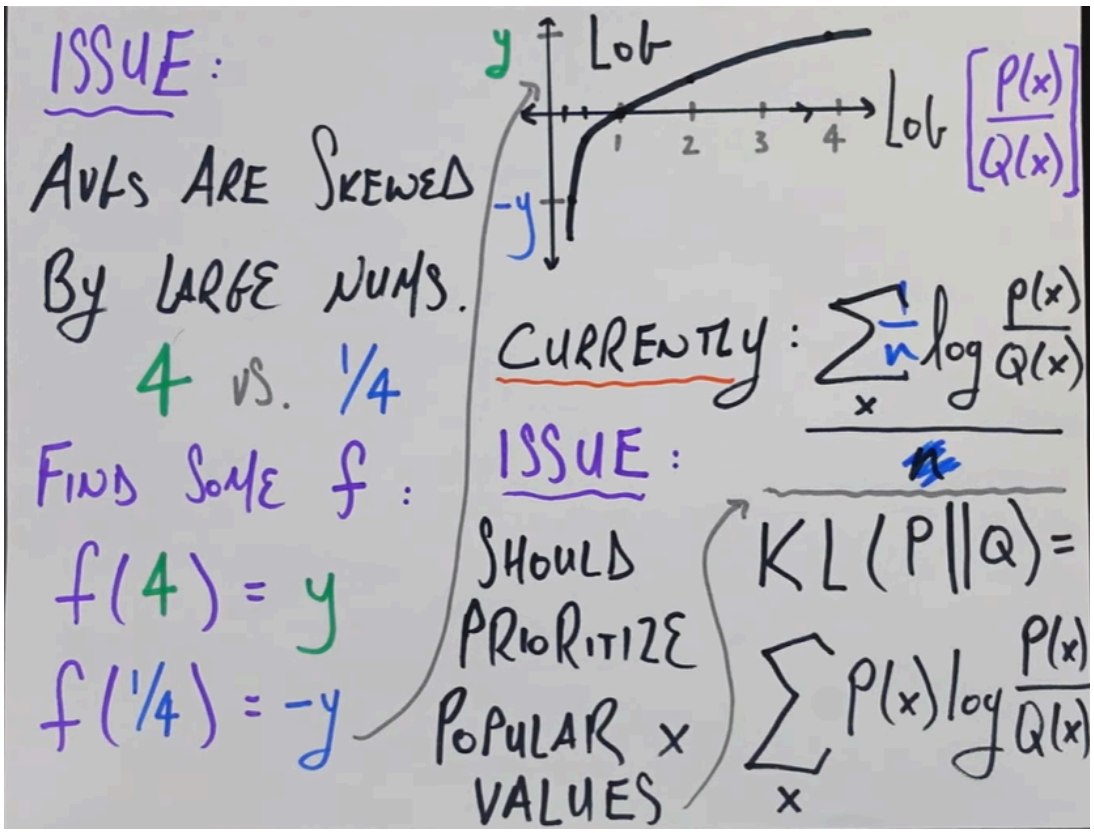
- Balancing the Losses: It's crucial to balance the reconstruction loss and the KL divergence. Too much weight on the KL divergence can lead to overly regularized models that fail to capture the complexity of the data.
- Learning Rate and Batch Size: These hyperparameters can significantly affect training stability and model performance.
- Model Complexity: The complexity of the encoder and decoder should match the complexity of the data. For instance, convolutional layers are often used for image data.

In practice, training a VAE requires careful tuning of these aspects to ensure that the model learns both a meaningful latent space and provides good reconstructions of the input data.

Loss function of VAEs

KL Divergence:





- The loss function for a Variational Autoencoder (VAE) consists of two main components: the reconstruction loss and the regularization term (typically the Kullback-Leibler (KL) divergence).

VAE's total loss can be given as:

$$\mathrm{L}(\phi, \theta, x) = (reconstruction - loss) + (regularization - term)$$

$$\mathrm{L}(\phi, \theta, x) = \frac{1}{N}\sum_{i=1}^{N}(X_i - \hat{X}_i)^2 + KL[G(Z_\mu, Z_\sigma), \mathcal{N}(0,1)]$$

- While the KL-divergence measures the divergence between a pair of probability distributions, in this case, the pair of distributions being the latent vector Z (sampled from Z_μ and Z_σ) and unit normal distribution $\mathcal{N}(0,1)$. KL-divergence ensures that the latent-variables are close to the standard normal distribution.
- KL divergence for VAEs is defined as follows:

$$D_{KL}[N(\mu, \sigma \parallel N(0,1)] = \frac{1}{2}\sum\left(1 + log\left(\sigma^2\right) - \mu^2 - \sigma^2\right)$$

- The total loss is the sum of these two components. Let's break down each part:

1. Reconstruction Loss:

The reconstruction loss measures how well the decoder can reconstruct the original input from the latent space. The specific form of the reconstruction loss depends on the nature of the data. Common choices include:

- **Mean Squared Error (MSE):**
  $\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2$
  where N is the number of data points, x_i is the original input, and $\hat{x}_i$ is the reconstructed output.
- **Binary Cross-Entropy (BCE):**
  $\mathrm{BCE} = -\frac{1}{N}\sum_{i=1}^{N}(x_i \log(\hat{x}_i) + (1 - x_i)\log(1 - \hat{x}_i))$
  where x_i is the original binary input and $\hat{x}_i$ is the reconstructed output.

- MSE is particularly well-suited for regression tasks, as it quantifies the average squared difference between predicted and actual target values.
- On the other hand, Cross Entropy Loss is more commonly used in classification problems, where it measures the divergence between the predicted probability distribution and the true distribution of target classes.
- Selecting the appropriate loss function for your model can significantly impact its performance and ability to generalize to unseen data.
- By considering the nature of the problem, the distribution of the data, and the desired properties of the model's predictions, you can choose the loss function that best aligns with your objectives.

2. Regularization Term (KL Divergence):

The regularization term encourages the latent space to follow a prior distribution, typically a standard normal distribution ($N(0, 1)$).

$$\text{KL Divergence} = -\frac{1}{2}\sum_{i=1}^{D}\left(1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2\right)$$

where D is the dimensionality of the latent space, and μ_i and σ_i are the mean and standard deviation of the Gaussian distribution produced by the encoder for each dimension.

3. Total Loss:

The total loss is the sum of the reconstruction loss and the KL divergence. The objective is to minimize this total loss during training.

$$\text{Total Loss} = \text{Reconstruction Loss} + \text{KL Divergence}$$

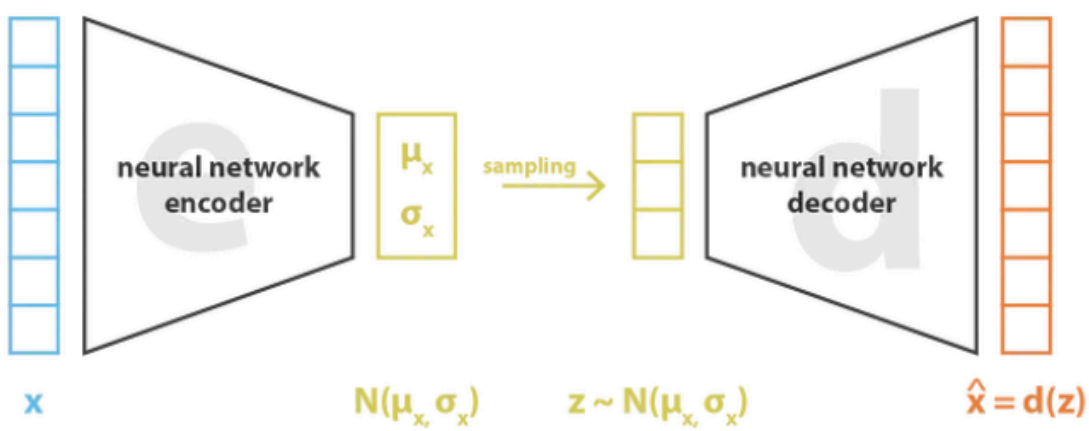
The balance between the reconstruction loss and KL divergence is controlled by a hyperparameter, usually denoted as β. The total loss can be modified as:

$$\text{Total Loss} = \text{Reconstruction Loss} + \beta \times \text{KL Divergence}$$

Adjusting β allows for trading off reconstruction accuracy and regularization strength.

During training, the goal is to minimize this total loss using optimization algorithms like stochastic gradient descent (SGD) or its variants (e.g., Adam).

In summary, the VAE loss function combines a reconstruction term to ensure accurate data reconstruction and a regularization term to shape the latent space distribution. The balance between these two components is crucial for effective learning and representation.



$$\text{loss} = \| x - \hat{x} \|^2 + KL[\, N(\mu_x, \sigma_x), N(0, I)\,] = \| x - d(z) \|^2 + KL[\, N(\mu_x, \sigma_x), N(0, I)\,]$$

In variational autoencoders, the loss function is composed of a reconstruction term (that makes the encoding-decoding scheme efficient) and a regularisation term (that makes the latent space regular).

Intuitions about the regularization

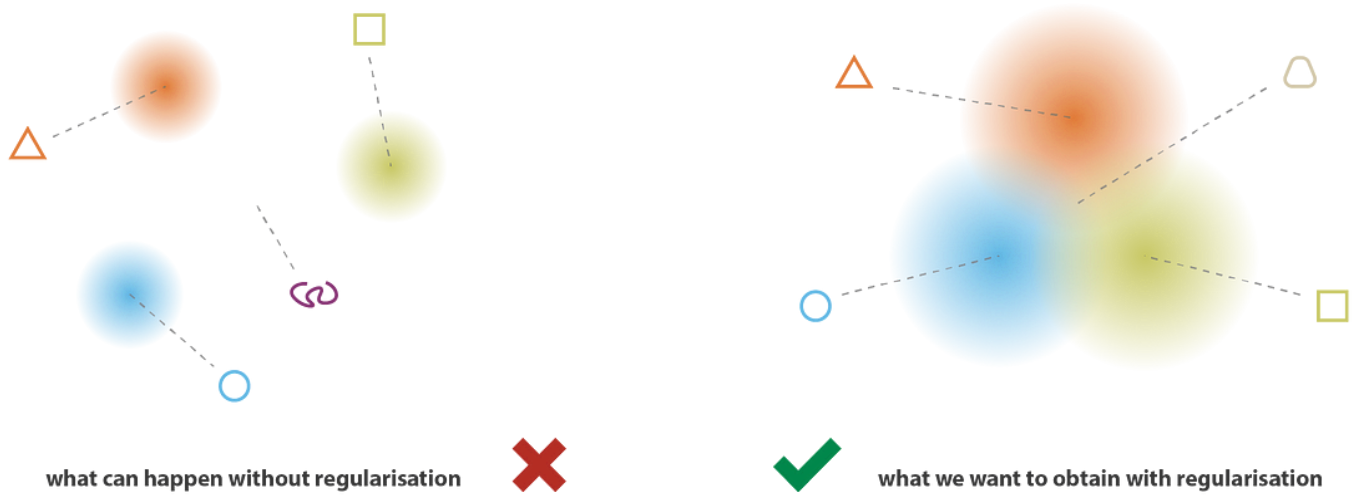
- The regularity that is expected from the latent space in order to make generative process possible can be expressed through two main properties:
    - **continuity** (two close points in the latent space should not give two completely different contents once decoded) and
    - **completeness** (for a chosen distribution, a point sampled from the latent space should give “meaningful” content once decoded).

Difference between a “regular” and an “irregular” latent space.

The only fact that VAEs encode inputs as distributions instead of simple points is not sufficient to ensure continuity and completeness. Without a well defined regularization term, the model can learn, in order to minimize its reconstruction error, **to “ignore” the fact that distributions are returned and behave almost like classic autoencoders** (leading to overfitting). To do so, the encoder can either return distributions with tiny variances (that would tend to be punctual distributions) or return distributions with very different means (that

would then be really far apart from each other in the latent space). In both cases, distributions are used the wrong way (cancelling the expected benefit) and continuity and/or completeness are not satisfied.

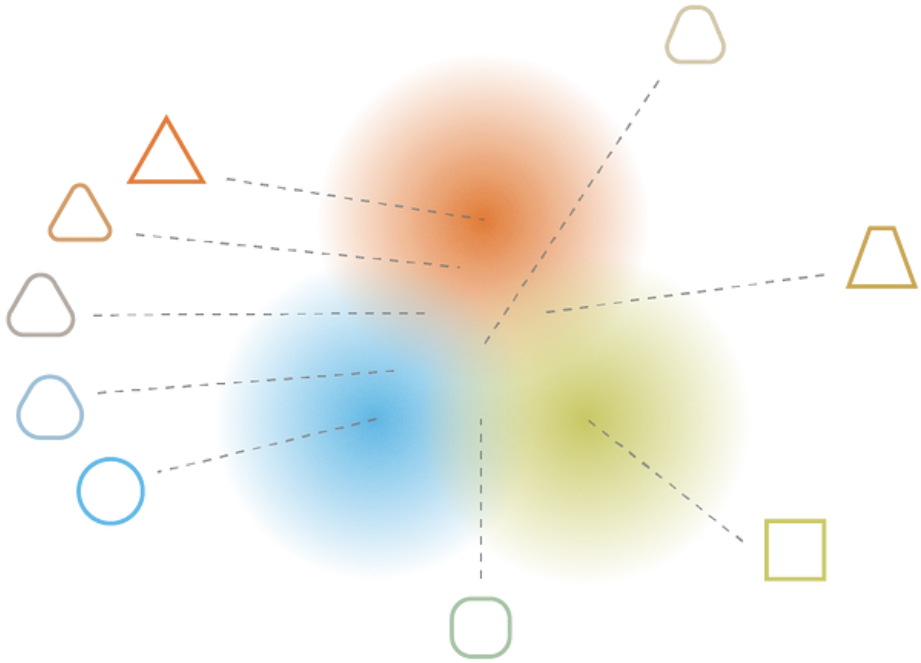
- So, in order to avoid these effects **we have to regularize both the covariance matrix and the mean of the distributions returned by the encoder**.
- In practice, this regularization is done by enforcing distributions to be close to a standard normal distribution (centered and reduced).
- This way, we require the covariance matrices to be close to the identity, preventing punctual distributions, and the mean to be close to 0(zero), preventing encoded distributions to be too far apart from each other.



The returned distributions of VAEs have to be regularized to obtain a latent space with good properties.

- With this regularization term, we prevent the model to encode data far apart in the latent space and encourage as much as possible returned distributions to “overlap”, satisfying this way the expected continuity and completeness conditions.
- Naturally, as for any regularization term, this comes at the price of a higher reconstruction error on the training data.
- The tradeoff between the reconstruction error and the KL Divergence can however be adjusted .

- We can observe that continuity and completeness obtained with regularization **tend to create a “gradient” over the information encoded in the latent space**.
- For example, a point of the latent space that would be halfway between the means of two encoded distributions coming from different training data should be decoded in something that is somewhere between the data that gave the first distribution and the data that gave the second distribution as it may be sampled by the autoencoder in both cases.

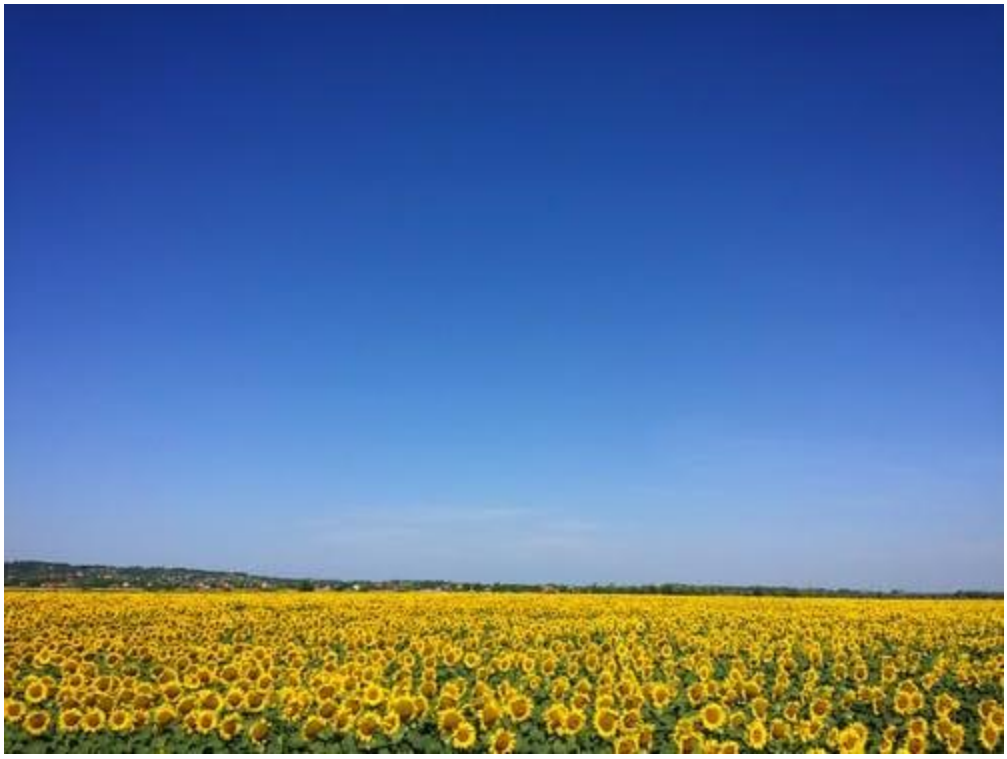


Regularization tends to create a “gradient” over the information encoded in the latent space.

Latent space representation and inference in VAEs

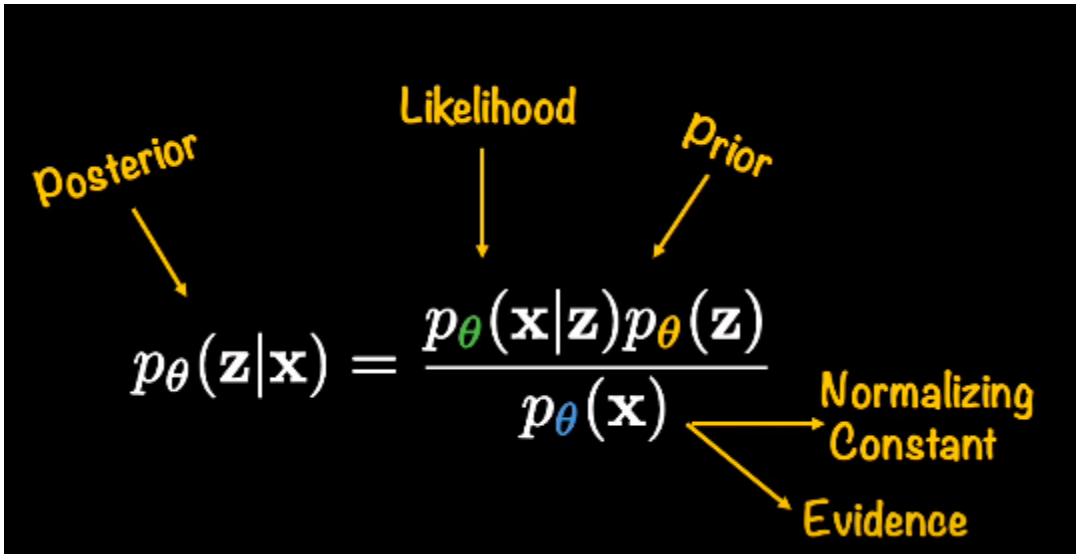
- In many interesting latent variable models, computing the posterior is not tractable.
- Variational inference provides a method to compute an approximate posterior distribution in such cases.
- ***Latent Variable:***
    - *Latent variables are the variables that are not directly observed but are inferred from other variables or we can say Latent variables allow us to describe very complex distributions over visible variables.*
    - *Let’s consider a rough **example**:*

- *have a look at the image shown below.*
- *Let’s suppose I’ve sent you this image to put it as wallpaper on screen.*
- *As we already know that the pixels in the image are dependent on the neighbors (because of texture color and all).*
- *Now you may ask why the sky is blue, not black? (because I've decided to show you daytime as opposed to night time).*
- *You can also ask why the sky is not cloudy? (because i’ve decided to show you a sunny day) and so on.*
- *So these decisions (daytime, sunny etc) are not explicitly known to you (they are hidden to you).*
- *Thus we only observe the image but what we observe depends on these latent (hidden) decisions.*



Variational Inference

- While studying simple models like Naive Bayes, Linear Regression, we **assume** that the data is from an underlying distribution, more likely from a normal distribution.
- Also, in Naive Bayes, there is an **assumption** that all the samples are iid (independent and identically distributed) but in reality, that’s not the case.



A latent variable model and Bayes rule to obtain the posterior distribution.

- In the above image, we have a latent variable model and Bayes rule to obtain the posterior distribution of the **latent variable z.**
- Examine the denominator p(x). This is called the evidence, and we can calculate it by marginalizing out the latent variables: **$p(x)=\int p(x|z)p(z)dz$.**
- Unfortunately, this integral **requires exponential time to compute** as it needs to be evaluated over all configurations of latent variables. We therefore need to **approximate this posterior distribution.**

➔ Variational Inference suggests us to use another **distribution q with parameters Φ** as an approximation to the posterior.

➔ The most famous algorithms for variational inference are <u>**Expectation-Maximization(EM)**</u> **algorithm** and **Variational Autoencoders.**

Variational inference approximates the posterior with a family of distributions $q_\lambda(z \mid x)$. The variational parameter λ indexes the family of distributions. For example, if q were Gaussian, it would be the mean and variance of the latent variables for each datapoint $\lambda_{x_i} = (\mu_{x_i}, \sigma^2_{x_i}))$.

How can we know how well our variational posterior $q(z \mid x)$ approximates the true posterior $p(z \mid x)$? We can use the Kullback-Leibler divergence, which measures the information lost when using q to approximate p (in units of nats):

$$\mathbb{KL}(q_\lambda(z \mid x) \mid\mid p(z \mid x)) =$$

$$\mathbf{E}_q[\log q_\lambda(z \mid x)] - \mathbf{E}_q[\log p(x, z)] + \log p(x)$$

Then we can use the variational inference algorithm to learn the variational parameters (gradient ascent on the ELBO to learn λ).

- VI is a way to approximate hard to compute posteriors $p(z|x) = \frac{p(x|z)p(z)}{\int p(x|z)p(z)dz}$
- The denominator ("evidence") is usually intractable
- In VI we place a variational distribution which we can control denoted by $q(z)$ and we optimize it's (variational) parameters θ to approximate the true posterior
- We take a specific metric which allows us to disregard the denominator, KL divergence, and by applying it to $KL(q(z)||p(z|x))$ we are able to only optimize the part here that depend on θ, which is called the ELBO (Evidence Lower BOund) which we want to maximize (or minimize the $-ELBO$)
  - $KL(q(z)||p(z|x)) = \int_z q(z) \log \frac{q(z)}{p(z|x)} dz = \mathbb{E}_{z\sim q}[\log q(z)] - \mathbb{E}_{z\sim q}[\log p(z|x)]$
  - Applying Bayes formula on the 2nd term we get:
  - $= \mathbb{E}[\log q(z)] - \mathbb{E}[\log p(x|z)p(z)] + \log p(x)$
  - $ELBO = \mathbb{E}_{z\sim q_\theta(z)}[\log p(x|z)p(z) - \log q_\theta(z)]$
- With some other approximating assumptions we can derive a coordinate-ascent VI (CAVI) [assuming "mean field"], or a simple enough (natural) gradient ascent VI [assuming exponential family].

2. Inference Objective (Variational Lower Bound):

- The goal is to maximize the Evidence Lower Bound (ELBO):
  $\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q(z|x)} [\log p(x|z; \theta)] - \mathrm{KL}(q(z|x)||p(z))$
  where:
  - θ are the parameters of the decoder (generative model),
  - ϕ are the parameters of the encoder,
  - $p(x|z; \theta)$ is the likelihood of reconstructing x given z,
  - $\mathrm{KL}(q(z|x)||p(z))$ is the Kullback-Leibler (KL) divergence between the approximate posterior $q(z|x)$ and the prior $p(z)$.

Applications of VAEs in Image Generation

Variational Autoencoders (VAEs) are widely used in **image generation** due to their ability to model complex distributions in a low-dimensional latent space and generate new samples by sampling from this space. Here are some key **applications of VAEs in image generation**:

1. Image Synthesis (Generation of New Images)

VAEs are generative models that can create new, unseen images by sampling from the learned latent space. After training on a dataset of images (e.g., faces, objects), VAEs can generate new images by sampling random points from the latent space and passing them through the decoder. These images are often coherent and capture the patterns learned from the training data.

- **Example**: A VAE trained on a dataset of faces can generate new, realistic faces by decoding random latent vectors from the latent space.

2. Image Interpolation

The latent space learned by a VAE is continuous, allowing for smooth interpolation between different images. By taking two different latent vectors corresponding to two different images and interpolating between them, VAEs can generate intermediate images that smoothly transition from one image to another.

- **Example**: Interpolating between two faces can generate a series of intermediate faces that blend characteristics of both, creating a morphing effect between the images.

3. Anomaly Detection in Images

VAEs can be used for **anomaly detection** in images by reconstructing normal images and comparing the reconstruction error for new data. Since the VAE is trained to model the distribution of normal data, it

struggles to accurately reconstruct images that are significantly different (anomalies). High reconstruction error can be used as an indicator of an anomaly.

- **Example**: In medical imaging, VAEs can be used to detect abnormalities by comparing the reconstruction error for normal images (e.g., healthy tissues) versus images with anomalies (e.g., tumors).

4. Image Denoising

VAEs can be applied to the task of image denoising by learning to reconstruct clean images from noisy ones. The encoder learns to map noisy images to the latent space, and the decoder reconstructs the clean image from this latent representation. The ability of VAEs to capture the underlying structure of the data helps in removing noise.

- **Example**: In image denoising, VAEs can be trained on pairs of noisy and clean images, allowing the model to learn how to remove noise while preserving important features.

5. Semi-Supervised Learning for Image Classification

VAEs can be used in **semi-supervised learning** settings, where only a small portion of the dataset has labeled images. The generative nature of VAEs allows them to learn a useful latent representation of the data from both labeled and unlabeled images. This latent representation can then be used to improve the performance of image classification tasks, even with limited labeled data.

- **Example**: In medical imaging, where obtaining labeled data can be expensive and time-consuming, VAEs can help improve classification accuracy by learning from a combination of labeled and unlabeled images.

6. Attribute-Based Image Manipulation

VAEs allow for **controllable image manipulation** by altering specific dimensions of the latent space that correspond to particular attributes. Once the VAE is trained, different attributes (such as rotation, lighting, or facial expressions) can be adjusted by moving along certain axes in the latent space.

- **Example**: In facial image generation, specific attributes like the smile or hair color of a person can be modified by manipulating the latent vector in controlled directions, resulting in edited images with desired attributes.

7. Super-Resolution Imaging

VAEs can be used to perform **super-resolution**, where low-resolution images are mapped to a latent space and then decoded into higher-resolution versions. This helps in generating detailed high-resolution images from compressed or blurry low-resolution images.

- **Example**: In satellite imagery or medical imaging, VAEs can enhance the resolution of images to help with finer details or analysis.

8. Creative Art Generation

VAEs can be applied in **creative art generation** by training them on datasets of artistic styles or patterns. They can generate unique and creative images by sampling from the latent space or blending various styles.

- **Example**: Artists and designers can use VAEs to generate novel designs, such as abstract paintings or digital art, based on training the model on existing styles and artworks.

9. Image Inpainting

VAEs can be used for **image inpainting**, where missing or corrupted parts of an image are filled in by the model. The VAE's encoder learns to model the entire image structure, and the decoder can fill in the missing pixels by generating plausible content based on the context provided by the rest of the image.

- **Example**: If a portion of a face image is missing or occluded, a VAE can infer and reconstruct the missing region, producing a complete and coherent image.

10. Style Transfer

VAEs can be used in **style transfer** tasks by learning separate representations for the content and style of images. This allows for the generation of images that combine the content of one image with the style of another.

- **Example**: VAEs can take the structure of one image (such as a photograph of a city) and combine it with the style of a painting, resulting in a new image that blends both.

11. Latent Space Visualization

VAEs allow for **latent space visualization**, where high-dimensional image data is compressed into a lower-dimensional latent space that can be visualized. This helps in understanding the distribution of data, discovering clusters, and identifying meaningful patterns in images.

- **Example**: Visualizing the latent space of a VAE trained on handwritten digits (like the MNIST dataset) can show how different digit classes are organized in the latent space and how interpolation between them corresponds to digit morphing.